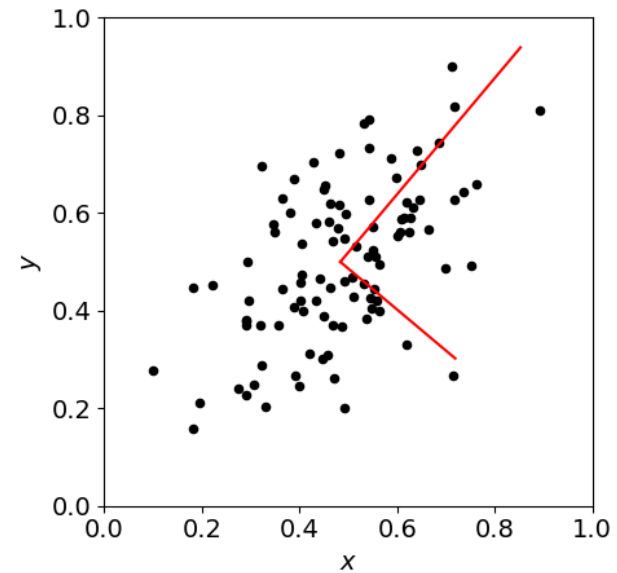
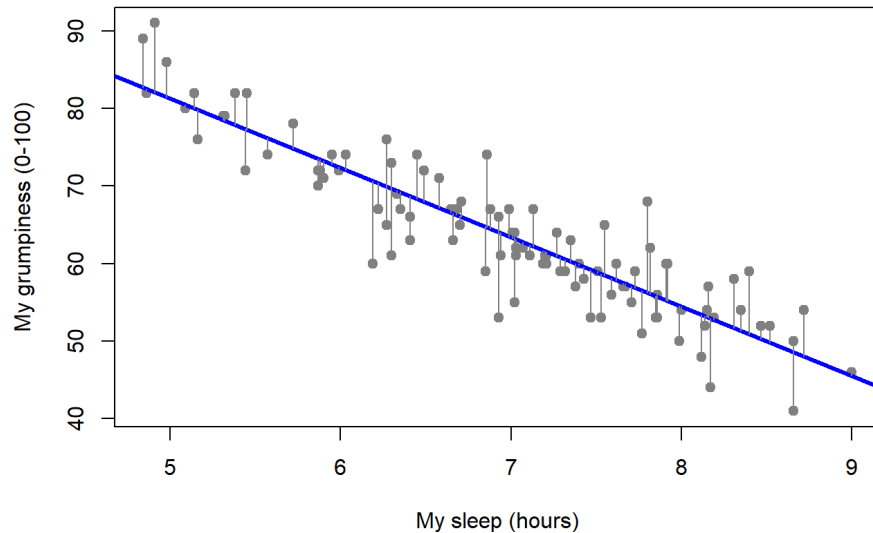
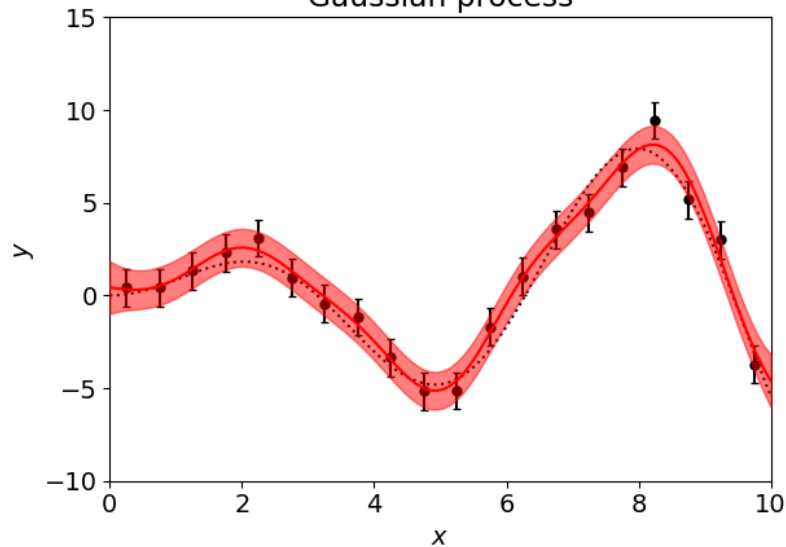


Class 4: Regression

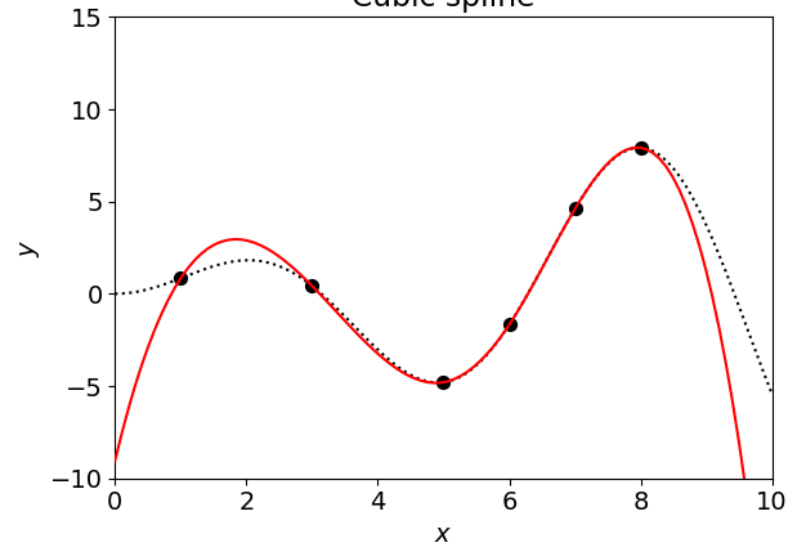
Regression Line Close to the Data



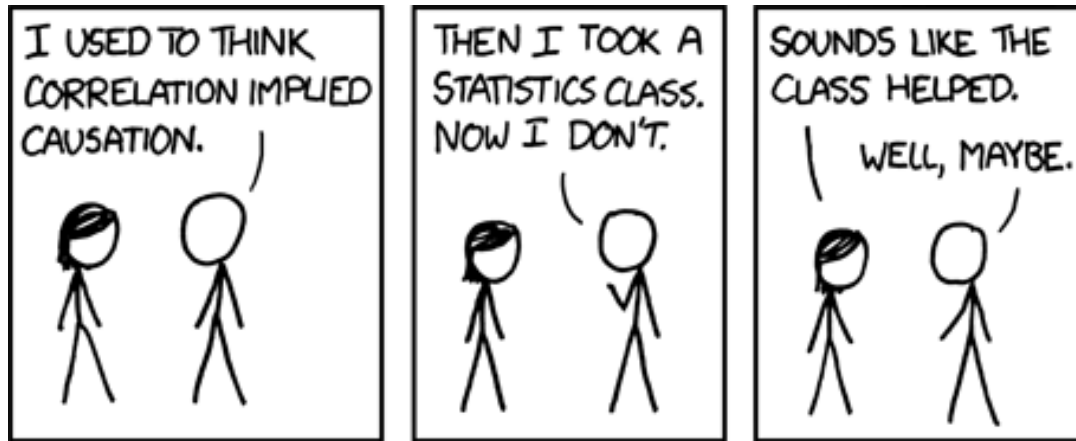
Gaussian process



Cubic spline



Class 4: Regression

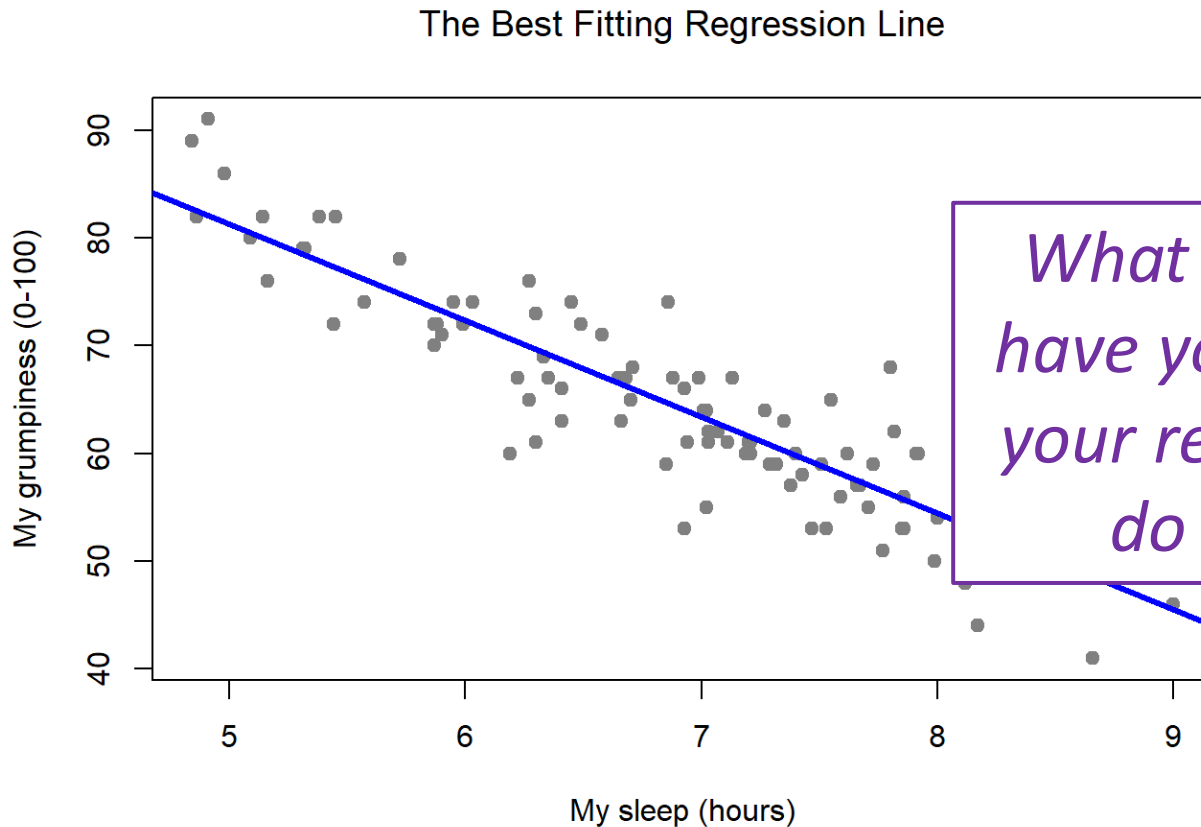


- Week 1: Probability and statistics
- Week 2: Correlation testing
- Week 3: Model fitting
- Week 4: Regression
- Week 5: Error estimates
- Week 6: Bayesian methods

Regression

“Regression” describes statistical methods which predict an **output variable** in terms of some **input variables**

Outcome (dependent) variable



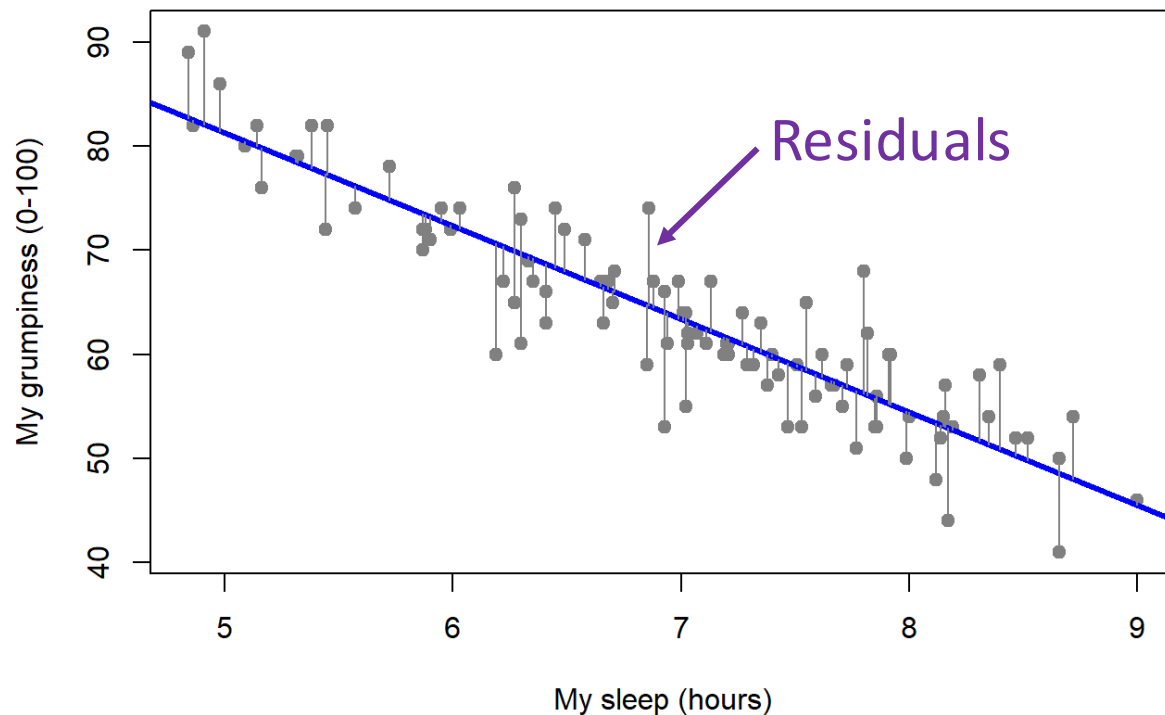
*What methods
have you used in
your research to
do this??*

Predictor (independent) variable

Linear least-squares regression

Linear regression determines a straight-line relationship by **minimising the residuals** between the line and points

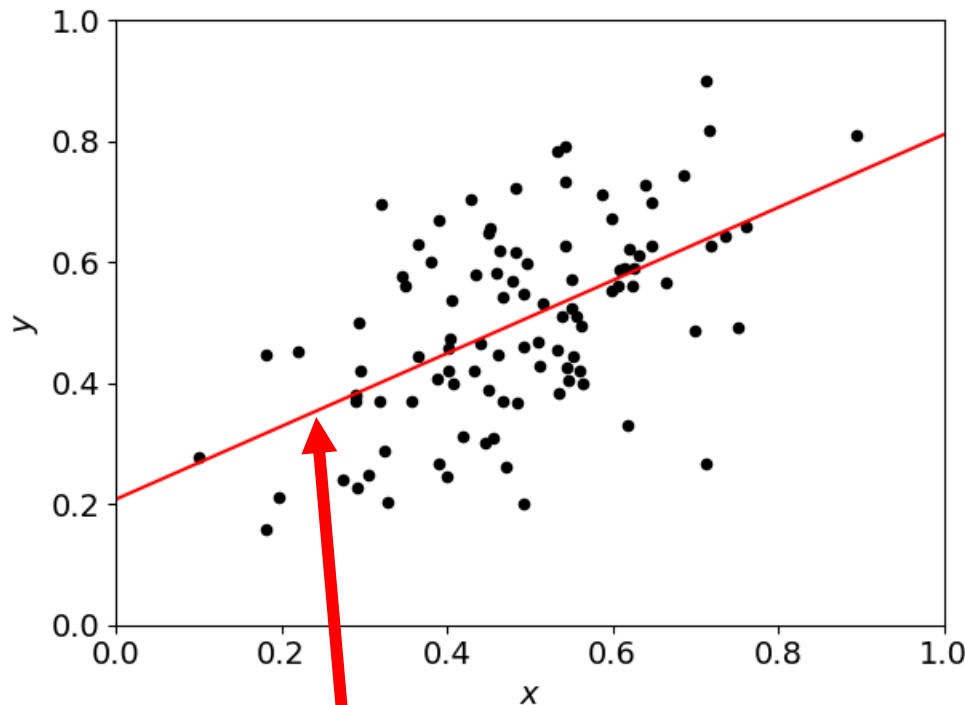
Regression Line Close to the Data



Notes:

- All points have equal weight
- No errors in the points are considered

Worked example: Least-squares regression

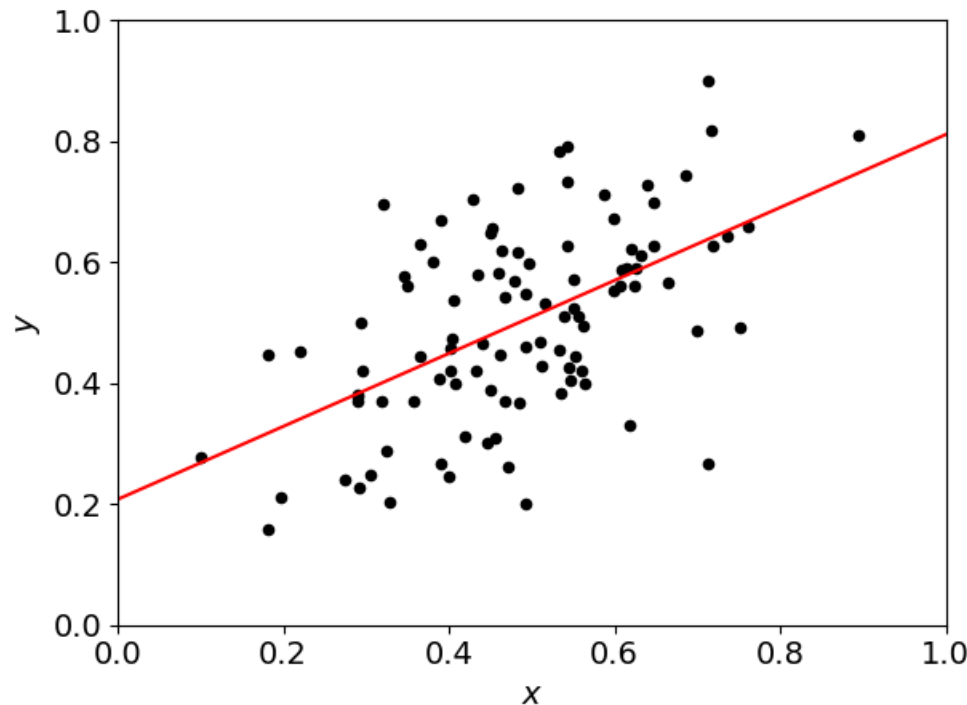


How “good” is the fit?

We describe the quality of the fit by the “**coefficient of determination**” (R^2) where R is the same as the Pearson correlation coefficient (r) we met in the previous class!

```
# Read in the example correlation dataset
# You will need to change the file path to the location where you've saved the data
data = np.loadtxt('../datasets/correlation_example.dat')
x,y = data[:,0],data[:,1]
# Perform linear regression on the dataset to determine the slope and intercept
result = stats.linregress(x,y)
a,b = result.slope,result.intercept
```

Worked example: Least-squares regression



$$R^2 = 1 - \frac{\sum_i (y_i - y_{\text{mod},i})^2}{\sum_i (y_i - \bar{y})^2}$$

Fluctuation of
data around
model

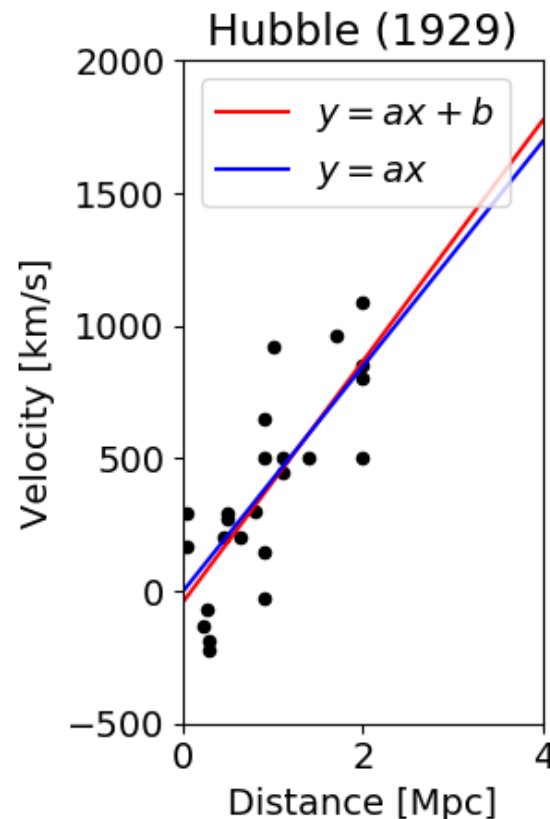
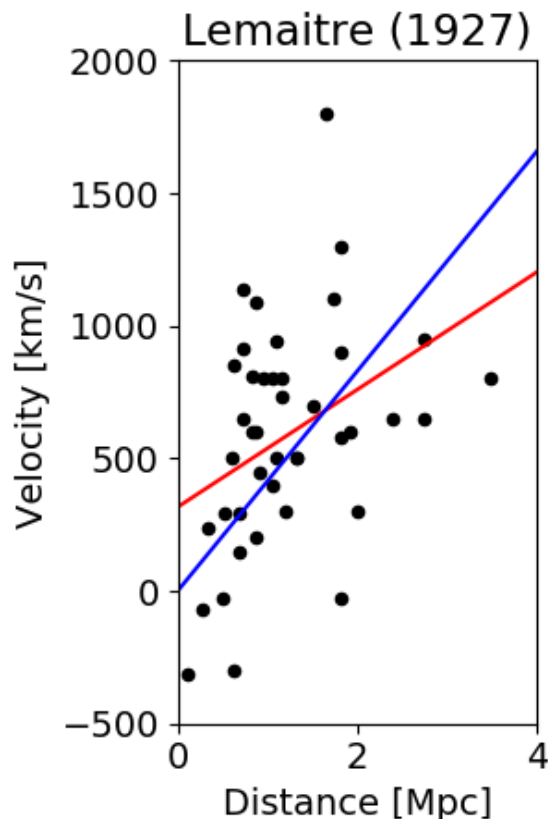
Fluctuation of
data around
its mean

We find $R^2 = 0.30$, i.e. “30% of the variance of the points is explained by the model fit”

```
# Read in the example correlation dataset
# You will need to change the file path to the location where you've saved the data
data = np.loadtxt('../datasets/correlation_example.dat')
x,y = data[:,0],data[:,1]
# Perform linear regression on the dataset to determine the slope and intercept
result = stats.linregress(x,y)
a,b = result.slope,result.intercept
```

Activity: Least-squares regression

- Returning to Hubble and Lemaitre's distance-velocity datasets, find the **linear least-squares regression lines** with and without an intercept, and the **value of R^2**



Lemaitre dataset

Slope = 221.7

Intercept = 316.5

Slope ($b = 0$) = 414.9

$R^2 = 0.14$

Hubble dataset

Slope = 453.8

Intercept = -40.4

Slope ($b = 0$) = 423.9

$R^2 = 0.62$

Activity: Least-squares regression

```
# Read in datasets
data = np.loadtxt('../datasets/lemaitre.dat')
dl,vl = data[:,0],data[:,1]
data = np.loadtxt('../datasets/hubble.dat')
dh,vh = data[:,0],data[:,1]

# Find linear regression line slope + intercept  $y = a*x + b$ 
result = stats.linregress(dl,vl)
al,b1 = result.slope,result.intercept
print('\nLemaitre dataset slope =',al)
print('Lemaitre dataset intercept =',b1)
result = stats.linregress(dh,vh)
ah,bh = result.slope,result.intercept
print('Hubble dataset slope =',ah)
print('Hubble dataset intercept =',bh)

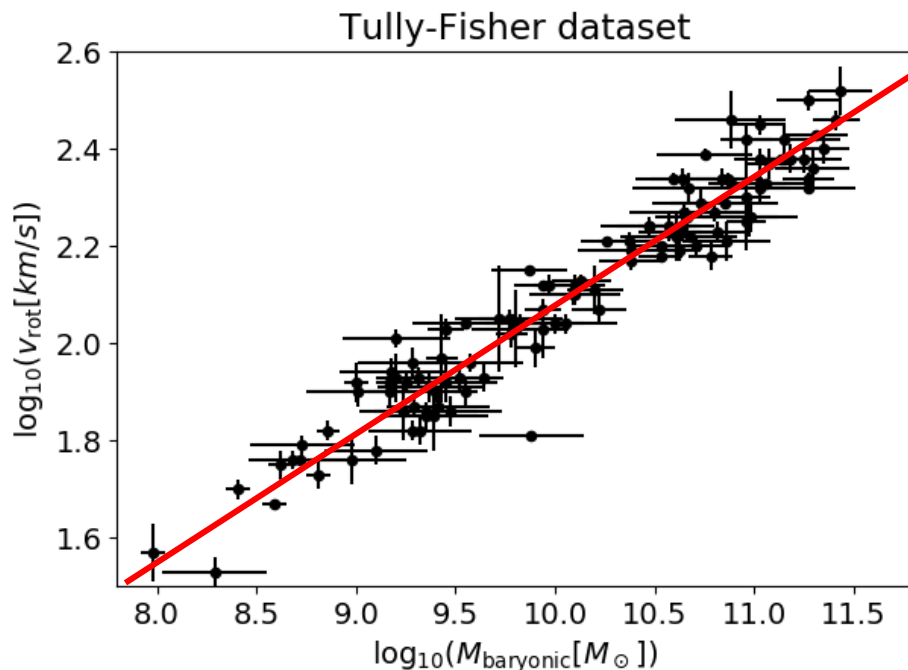
# Find linear regression line without intercept,  $y = a*x$ 
al2 = np.sum(dl*vl)/np.sum(dl**2)
print('\nLemaitre dataset slope (no intercept) =',al2)
ah2 = np.sum(dh*vh)/np.sum(dh**2)
print('Hubble dataset slope (no intercept) =',ah2)

# Check  $R^2$  value
r2l = 1 - np.sum((vl-(al*dl+b1))**2)/np.sum((vl-np.mean(vl))**2)
print('\nLemaitre dataset  $R^2$  =',r2l)
r2h = 1 - np.sum((vh-(ah*dh+bh))**2)/np.sum((vh-np.mean(vh))**2)
print('Hubble dataset  $R^2$  =',r2h)
```

Weighted regression

We can also **weight each point** when minimising the residuals from the line (this is just like the χ^2 statistic)

It's relatively common to have **errors in both co-ordinates**. In this case we could minimize the function:

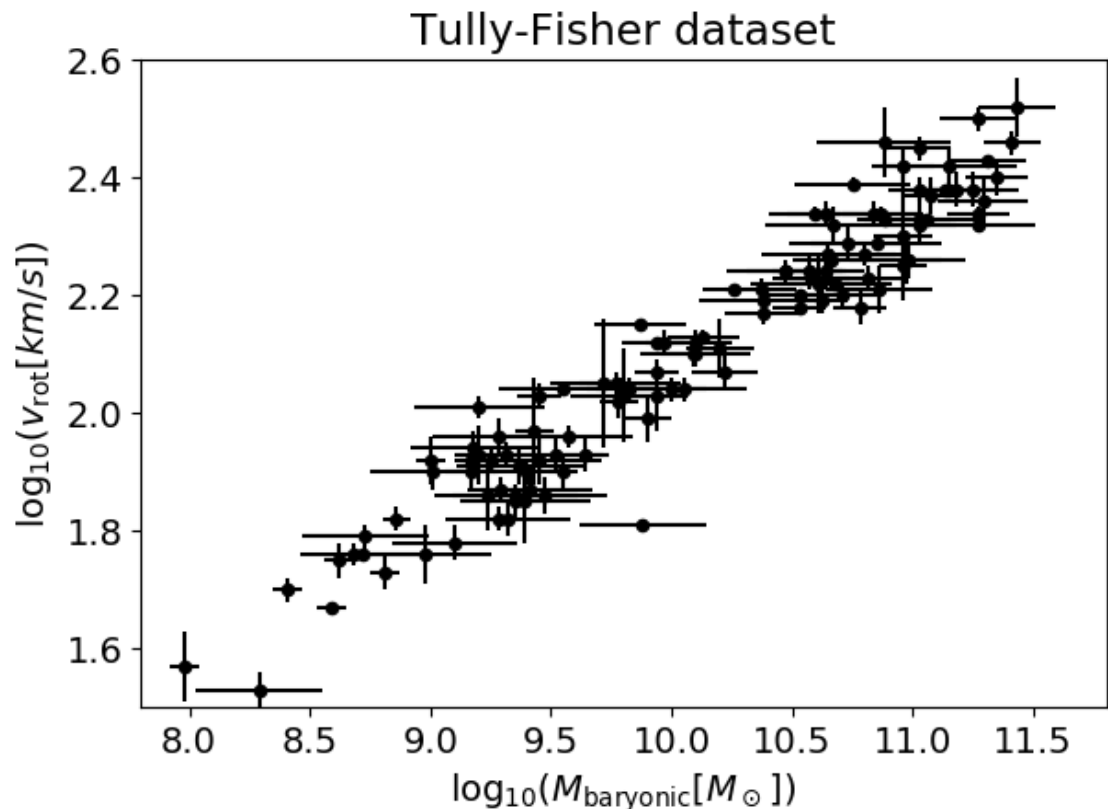


$$\chi^2(a, b) = \sum_{i=1}^N \frac{(y_i - ax_i - b)^2}{\sigma_{y,i}^2 + a^2 \sigma_{x,i}^2}$$

Error propagation: if $z = y - ax - b$,
then $\sigma_z^2 = \sigma_y^2 + a^2 \sigma_x^2$

Activity: the Tully-Fisher relation

- For example, consider an example dataset containing the stellar masses and rotation velocities of galaxies:
- Find the **best-fitting linear regression** by minimizing the function on the previous slide using the errors in both co-ordinates



Activity: the Tully-Fisher relation

- My solution: $a = 0.267$, $b = 0.00288$

```
# Function to minimise
def chisqtullyfisher(p,x,xerr,y,yerr):
    a,b = p[0],p[1]
    x1,y1 = x-np.mean(x),y-np.mean(y)
    chisq = np.sum((y1 - (a*x1 + b))**2/(yerr**2 + (a**2)*(xerr**2)))
    return chisq

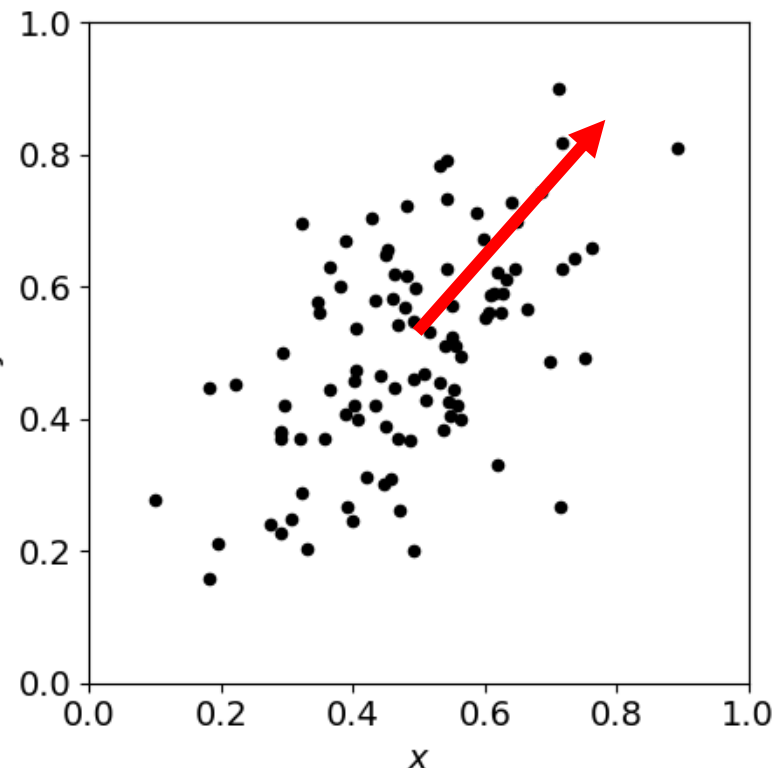
# Read in dataset
data = np.loadtxt('../datasets/tullyfisher.dat')
m,merr,v,verr = data[:,0],data[:,1],data[:,2],data[:,3]
n = len(m)

# Run minimisation
p0 = [0.25,0.]
result = minimize(chisqtullyfisher,p0,args=(m,merr,v,verr))
a,b = result['x']
```

Principal component analysis

Principal component analysis (PCA) identifies which variable combinations capture most information about the data

- **Geometrically**, PCA identifies the directions in which the cloud of variables is most elongated
- **Mathematically**, PCA determines the *eigenvectors* of the covariance matrix and sorts them in importance according to their corresponding *eigenvalues*



Worked example: PCA

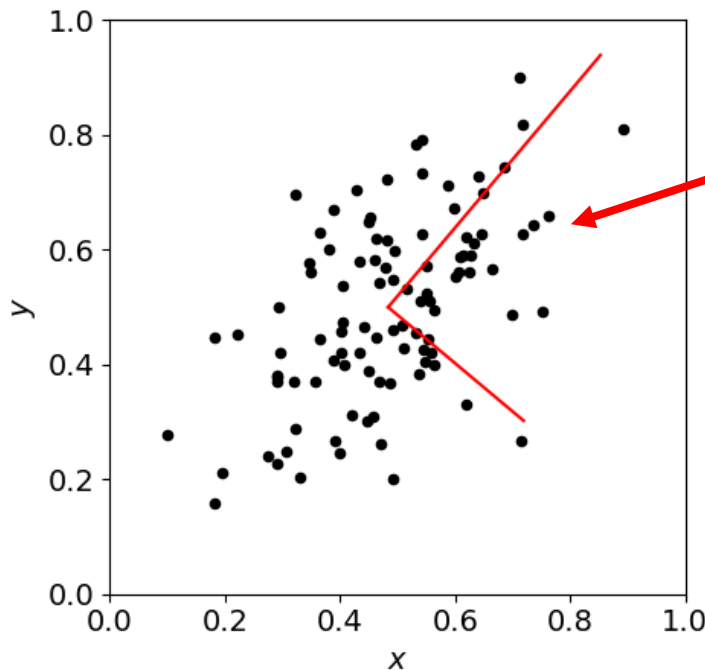
- Based on the (x, y) dataset on the previous slide:

- Find the **covariance matrix** of (x, y) : $C = \begin{pmatrix} 0.021 & 0.013 \\ 0.013 & 0.026 \end{pmatrix}$

$$C = \begin{pmatrix} \text{Cov}(x, x) & \text{Cov}(x, y) \\ \text{Cov}(y, x) & \text{Cov}(y, y) \end{pmatrix} \quad \text{Cov}(x, y) = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})$$

- Determine the **eigenvalues** and **eigenvectors** of C :
eigenvalues are $\lambda_1 = 0.037$, $\lambda_2 = 0.011$ with corresponding eigenvectors $\vec{v}_1 = (0.64, 0.77)$ and $\vec{v}_2 = (0.77, -0.64)$
- Express the data points in the **basis of the eigenvectors** –
new co-ordinates are (PC_1, PC_2) such that $\vec{x} = (x, y) = (\bar{x}, \bar{y}) + PC_1 \vec{v}_1 + PC_2 \vec{v}_2$

Worked example: PCA



The PCA coefficients correspond to a “rotated co-ordinate system” with these axes

All this conveniently happening within scikit-learn’s PCA!

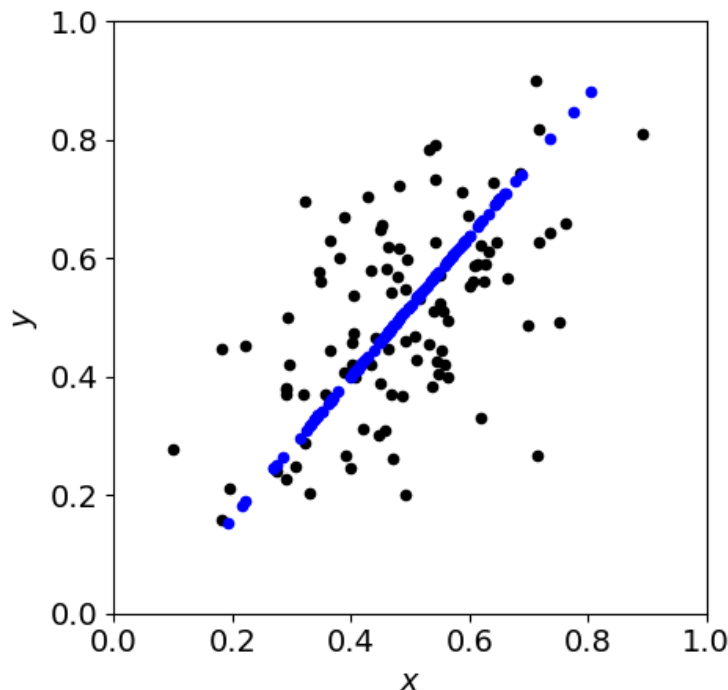
```
from sklearn.decomposition import PCA

# Determine the eigenvalues and eigenvectors of the example correlation dataset
pca = PCA(n_components=2)
data = np.transpose(np.array([x,y]))
pca.fit(data)
eigval = pca.explained_variance_
eigvec = pca.components_
l1,l2 = eigval[0],eigval[1]
v1,v2 = eigvec[:,0],eigvec[:,1]
print('Eigenvalue 1 =',l1,'eigenvector 1 =',v1)
print('Eigenvalue 2 =',l2,'eigenvector 2 =',v2)
```

Worked example: PCA

- PCA is commonly used for **dimensionality reduction**, i.e. approximating a dataset with a fewer number of variables

We can illustrate this by reconstructing our previous dataset using only 1 principal component, $(x, y) = (\bar{x}, \bar{y}) + \textcolor{blue}{PC}_1 \vec{v}_1$



- The blue points are an approximation of the original black points
- The amount of variance retained is determined by the size of the first eigenvalue λ_1 compared to the sum $\sum_i \lambda_i$

Activity: PCA

- Perform a Principal Component Analysis on the provided dataset of SDSS quasar magnitudes. How many principal components are needed to explain 90% of the variance?



Activity: PCA

```
# Read in dataset
infile = '../datasets/SDSS_quasar.dat'
df = pd.read_table(infile,delim_whitespace=True,header=2)
# Let's just use the 5 optical magnitudes for this test
umag = df['u_mag'].values
gmag = df['g_mag'].values
rmag = df['r_mag'].values
imag = df['i_mag'].values
zmag = df['z_mag'].values

# Determine the covariance between the 5 arrays
magarr = np.stack((umag,gmag,rmag,imag,zmag))
cov = np.cov(np.stack(magarr))
# This produces a 5x5 matrix as an outcome
print('Covariance =',cov)

# Determine the eigenvalues and eigenvectors
eigval,eigvec = np.linalg.eig(cov)
# Sort the eigenvalues in decreasing order
i = np.flip(np.argsort(eigval))
eigval,eigvec = eigval[i],eigvec[:,i]
print('Eigenvalues =',eigval)

# Determine the amount of variance retained
var = np.cumsum(eigval)/np.sum(eigval)
print('Variance retained =',var)
```

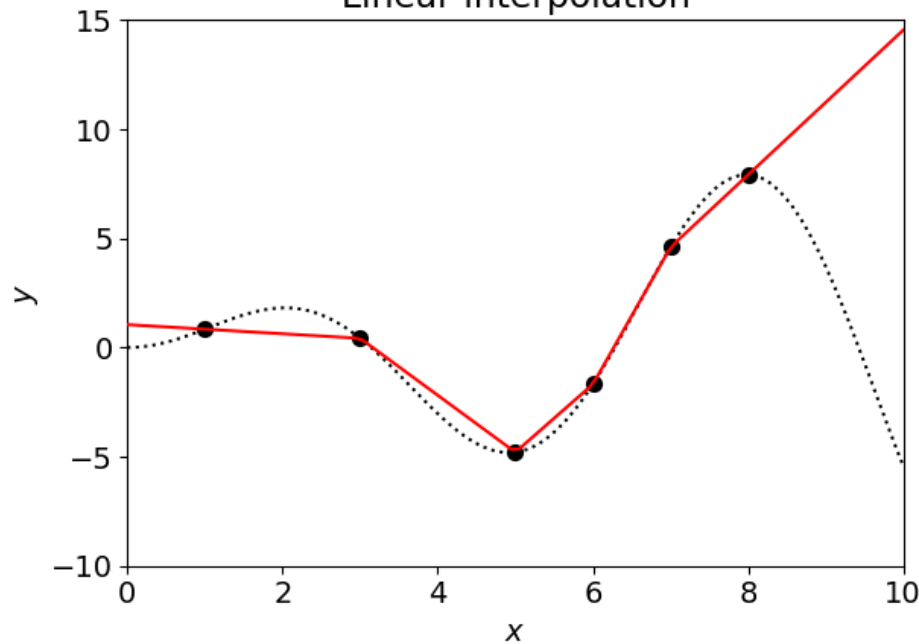
Fraction of variance from first
 N components =
(0.849, 0.977, 0.994, 0.998, 1.)

*How do we interpret
these values?*

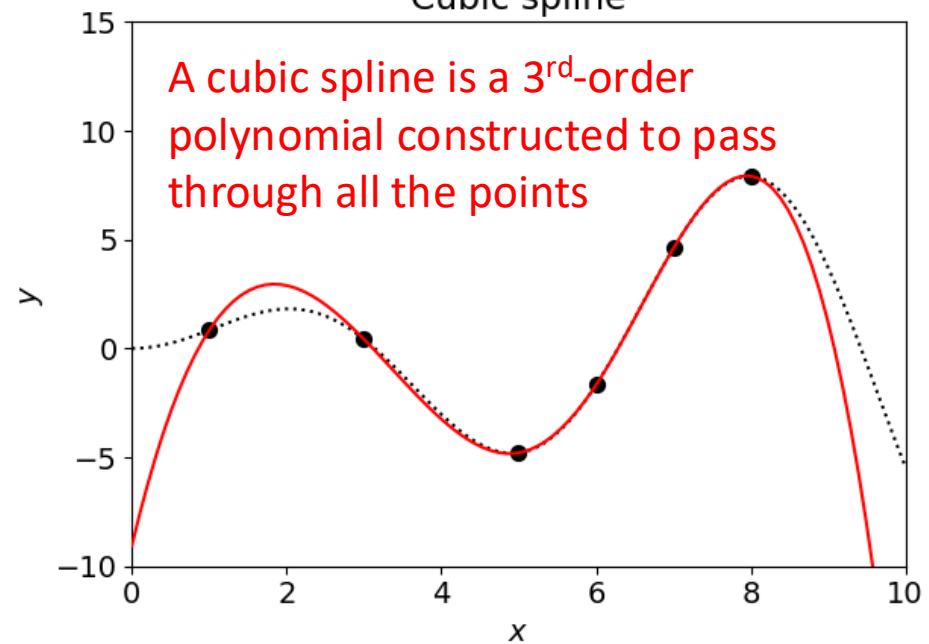
Interpolation

Interpolation involves predicting the outcome values in between the positions of our data points

Linear interpolation



Cubic spline

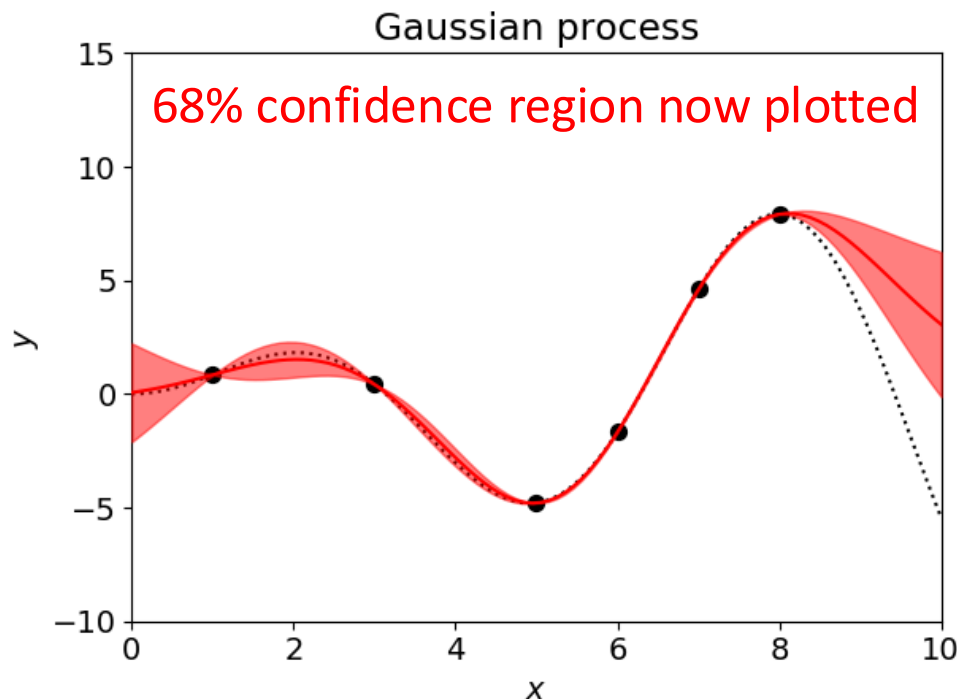


Note: these approaches don't provide an error in the interpolation

Gaussian Process

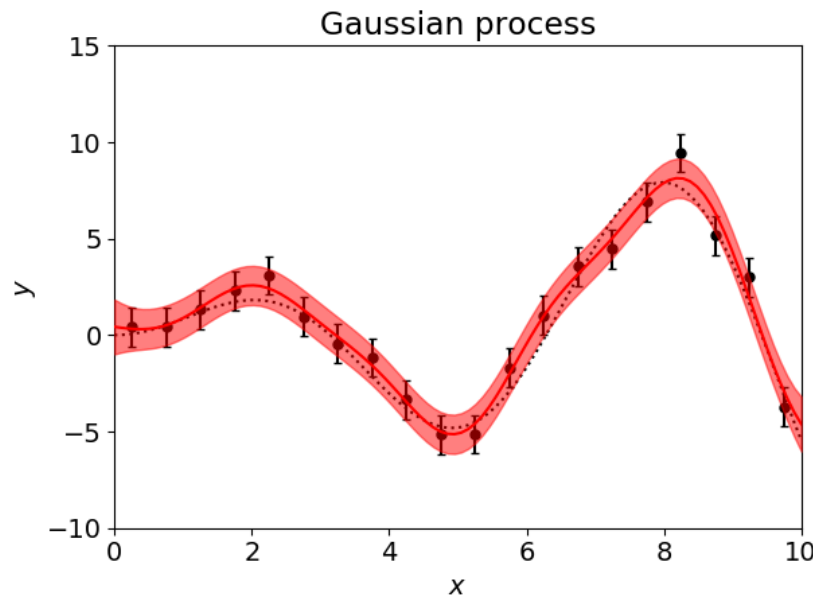
Interpolation with errors can be achieved by modelling the data using a “**Gaussian Process**” (also known as **kriging**)

In so doing, we’re imposing a statistical model for the correlations in the function (a “smoothness prior”)



- The Gaussian Process requires us to specify a “kernel” which describes the degree of correlation which is allowed in the function
- Here we have assumed $K(x_1, x_2) = e^{-\frac{1}{2}\left(\frac{x_1 - x_2}{L}\right)^2}$ where $L = 1$; this is a length scale of allowed variation

Worked example: Gaussian Process

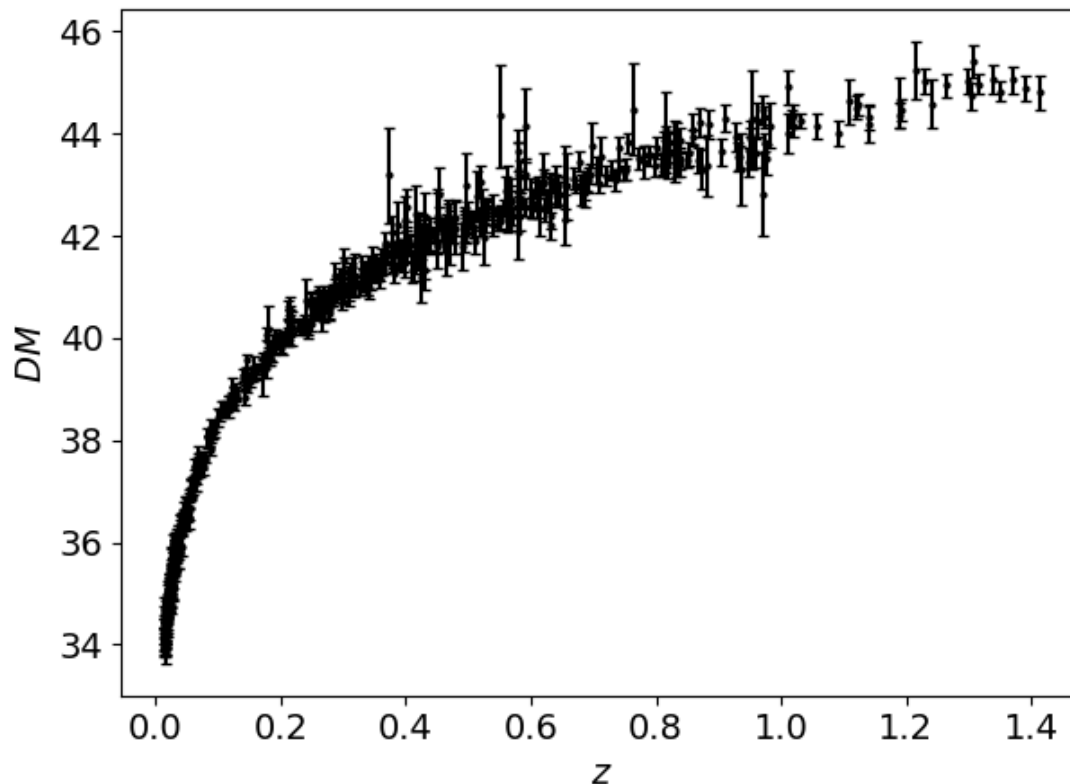


Kernel for Gaussian Process contains a “correlation term” and a “noise term”

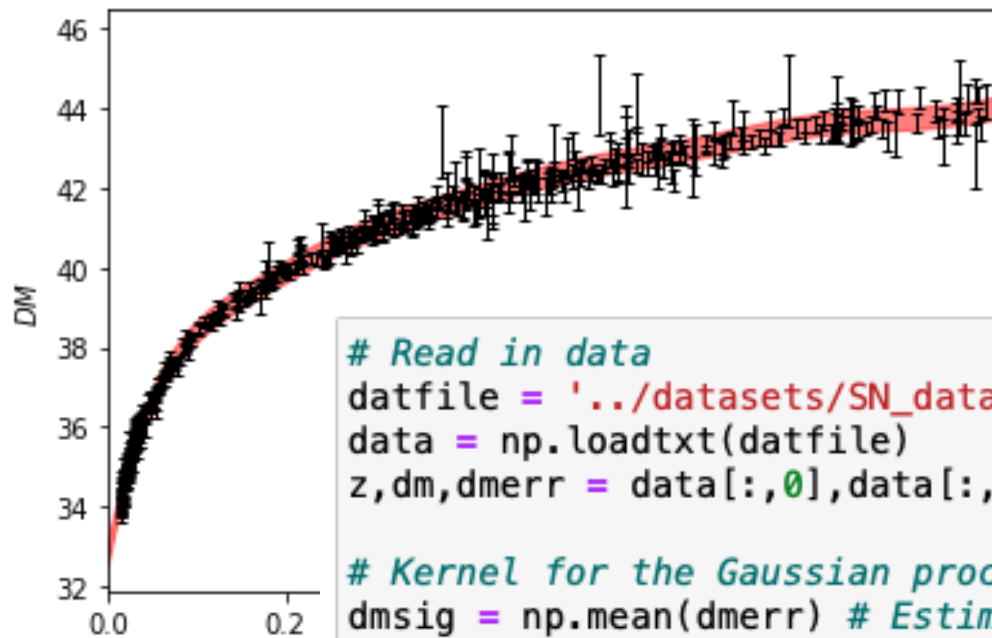
```
# Create a noisy dataset sampled from the model  $y = x * \sin x$ 
sig = 1. # value of standard deviation
nx = 20 # number of data points to sample
dx = (xmax-xmin)/nx
xn = np.linspace(xmin+0.5*dx,xmax-0.5*dx,nx) # x-values of the data points
yn = xn*np.sin(xn) + sig*np.random.normal(size=nx) # y-values are the model plus noise
# Kernel for the Gaussian process including noise (which is defined as a variance)
kernel = ConstantKernel(1.)*RBF(1.) + WhiteKernel(sig**2)
# Create the Gaussian process
gp = GaussianProcessRegressor(kernel=kernel,n_restarts_optimizer=10)
xn2 = xn.reshape(-1,1)
gp.fit(xn2,yn)
# Sample the Gaussian process and confidence interval at each value of x
xmod2 = xmod.reshape(-1,1)
ygp,ygperr = gp.predict(xmod2,return_std=True)
```

Activity: Gaussian Process

- Let's return to the **supernova distance-redshift dataset** from Class 3. *Fit a Gaussian process model to this dataset to predict the distance modulus and its error at any redshift.*



Activity: Gaussian Process



```
# Read in data
datfile = '../datasets/SN_dataset.dat'
data = np.loadtxt(datfile)
z, dm, dmerr = data[:,0], data[:,1], data[:,2]

# Kernel for the Gaussian process including noise
dmsig = np.mean(dmerr) # Estimate of noise value
kernel = ConstantKernel(1.)*RBF(1.) + WhiteKernel(dmsig**2)
gp = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=10)

# Fit the Gaussian process to the data
z2 = z.reshape(-1,1)
gp.fit(z2, dm)

# Sample the Gaussian process on a redshift grid
zmin, zmax, nz = 0., 1., 200
zmod = np.linspace(zmin, zmax, nz)
zmod2 = zmod.reshape(-1,1)
dmgp, dmgperr = gp.predict(zmod2, return_std=True)
```

Next week: Error Estimates

